Day 57 – API Management (APIM) – Part 6: OAuth Flow, JWT, SAML & Threat Protection

Class date: 14th September 2026 | Module: SAP API Management – OAuth2 credentials, JWT, SAML Assertion, XML/JSON Threat Protection

---

One correction to your JWT notes: you wrote that a JWT consists of **"header, payload, secret key."** The third part of an actual JWT is the **Signature**, not the Secret Key itself. The Secret Key (or a private key, for certificate-based signing) is the *input* used to mathematically *generate* that Signature – it is never stored as a literal, readable part of the token. So the correct structure is: **Header . Payload . Signature** (three dot-separated, Base64URL-encoded parts), where the Signature is produced by signing Header+Payload with the Secret Key. Corrected in Section 5 below. Everything else in today's notes was accurate.

1. How do you get an OAuth token for APIM? (start here first)

In CPI: you supply credentials directly → CPI authenticates → CPI is called.

But in APIM, it works differently: APIM is called first, and authentication/authorization then happens **based on the Policies** attached to that API Proxy – only once those policies pass is the call considered authenticated.

In APIM, if you want to generate any token, you need Policies. Basically, **in APIM, every single action works under Policies** – nothing happens automatically without one attached.

Practical step: **Create a generic API Proxy → attach the "Generate Access Token" policy (OAuth v2.0).**

Where do the credentials actually come from?

Recall: **Developer Hub** is where all the APIs are managed.

```
Developer Hub -> My Workspace -> Create an App -> Assign a Product to this App ->
you will get: API Key + API Secret (these act as the Client ID + Client Secret)
```

2. Validate – OAuth Token → Verify Access Token

Demo: Postman → call the endpoint → **POST** → pass the token as a **Bearer token.**

An **OAuth token** is tied to a specific **Application** – it was issued because that Application was registered and assigned a Product, as covered above.

APIM → Authentication → OAuth 2 (important): in practice, **OAuth2 is mostly used only in Real-Time** scenarios – where security genuinely needs to be dynamic rather than a long-lived static credential.

3. Custom attributes on a token – Set vs. Get

If you want a token **with custom attributes** attached to it, you use the **"OAuth v2.0 SET"** policy (confirms the naming from Day 56 – this is the policy that *sets* custom attributes onto an existing access token).

To then **retrieve/get** those attributes (or general token info) back out, you use the **"OAuth v2.0 Get"** policy.

4. JWT – JSON Web Token

JWT = JSON Web Token. JSON = JavaScript Object Notation – structured as **key-value pairs.**

Generate JWT policy – sets claims such as: **Expiration, Issuer, Subject, Audience**, and signs the token using a **Secret Key** (or certificate-based private key).

Two related, separate policies: **Decode JWT Token** (reads the Header and Payload back out, without checking the signature) and **Verify JWT Token** (fully checks the Signature as well, confirming the token is both readable *and* genuinely untampered).

JWT structure (corrected)

```
Header . Payload . Signature
```

- **Header** – metadata: token type (JWT) and the signing algorithm used (e.g. HS256, RS256).
- **Payload** – the actual claims/key-value data (Expiration, Issuer, Subject, Audience, etc.).
- **Signature** – generated by signing the encoded Header+Payload *using* the Secret Key (or a certificate's private key). This is what makes the token tamper-proof – the Secret Key itself is never part of the token.

Secret Key via a Certificate: instead of a plain shared-secret string, the signing key can also come from a **Certificate**, configured under a **Keystore section** – APIM reads that certificate and uses it to generate the token (JWT).

5. SAML Assertion Validation / Generation

Context: **SuccessFactors** → **SAML (OAuth2)** – configured via a ready-made **Policy Template.**

Scenario:

APIM → **SuccessFactors token**, requiring: **Client ID, Client Secret, SAML Content** – this overall pattern is called **OAuth2 SAML Bearer Assertion.**

1. **1st call:** generate the **SAML Content** (the SAML Assertion itself), via an OAuth/IdP endpoint.
2. **2nd call:** pass that generated SAML Content on to SAP APIM/SuccessFactors to exchange it for the real access token.

(This is the same two-call OAuth2 SAML Bearer Assertion flow already confirmed on Day 54 – get a SAML Assertion first, then exchange it for an OAuth Bearer token – just now viewed from the APIM-policy side rather than the CPI iFlow side.)

The two policies involved: **SAML Assertion Generation** (configuration) and **SAML Assertion Validation.**

6. XML Threat Protection

XML Threat Protection – something like **XML Content Validation.**

When it applies: based on the request/response **Headers** → **Content-Type** (e.g. **text/plain** or an XML content type) – it checks whether the **received or sent XML content is correct or not.**

- **XSD** → defines the formal **structure** of the XML (used for true schema-based validation).
- **XML Threat Protection** → checks whether the XML is **well-formed/proper**, even **without needing a full XSD** – a lighter-weight structural safeguard rather than full schema validation.

What it actually checks: whether the incoming XML's **format matches** configured limits or not – specifically: **how many fields, how many (child) nodes, and how large each field's value is allowed to be. If validation fails, it throws an error** and the request is rejected.

Example: a limit of **10 child nodes** configured – any XML payload arriving with **more** than 10 child nodes **fails.**

Scenario: only the Source System is supposed to send **5 fields.** If it sends more than that, the call **should fail** (be rejected).

Configurable limits in XML Threat Protection:

- **Field Names** – how many fields are allowed, e.g. up to 10.
- **Value** – how much length each field's value is allowed to have.
- **Child Nodes** – how many child nodes are allowed to be received.

Example: field **EmpID limited to 5 characters** – this is exactly the kind of **limitation option** XML Threat Protection lets you enforce.

7. JSON Threat Protection

JSON Threat Protection works the **same way as XML Threat Protection**, just for JSON content: **JSON Content** → **Validation** → **JSON Threat Protection.**

Demo: turned **on** XML Threat Protection, tested via **Postman**, and confirmed the demo worked – this kind of policy amounts to a **security-level validation**, analyzing each and every field going through XML Threat Protection.

Quick-recall summary

- In APIM, **every action runs under a Policy** – APIM itself is called first, then policies decide authentication.
- Credentials come from **Developer Hub** → **My Workspace** → **create an App** → **assign a Product**, yielding an **API Key + API Secret** (= Client ID + Client Secret).
- **OAuth v2.0 SET** = set custom attributes on a token; **OAuth v2.0 Get** = retrieve them.
- **JWT = Header.Payload.Signature** (not "Secret Key" as a literal part) – the Secret Key/certificate is only the signing input.
- **SuccessFactors OAuth2 SAML Bearer Assertion** = 2 calls: generate SAML Content, then exchange it for a token – handled by the SAML Assertion Generation/Validation policies.
- **XML/JSON Threat Protection** = structural safeguards (field count, node count, value length) that reject malformed/oversized payloads, even without a full XSD.